Software Design Analysis Report

A. Project Details

Project title: Digital Wallet Management System (DWMS)

Team / roll numbers: Group 5 (Financial Technology & Distributed Ledgers Dept.)

Name	Roll number
Adarsh Singh	202401100200017
Adarsh Yadav	202401100200018
Aditi Parashar	202401100200019
Aditya	202401100200020

Project objective: To provide a distributed, cryptographically fortified platform for real-time stored-value custody, peer-to-peer transfers, and double-entry ledger auditing. It guarantees zero balance drift and sub-second transaction finality via two-phase concurrency control and strict PCI-DSS compliance.

Technology stack: React Native / TypeScript, Node.js & Express, PostgreSQL 16+, Redis Enterprise, Apache Kafka, Argon2id & AES-256-GCM, Docker & Kubernetes

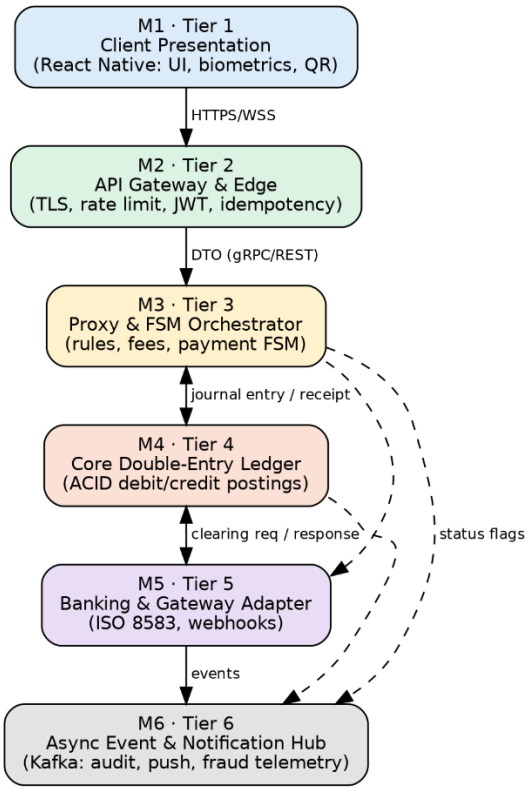
B. Module Identification and Modularization

Total number of modules: 6. The architecture decomposes presentation, security/routing, proxy handling, core double-entry accounting, banking integration, and asynchronous telemetry across distinct, loosely coupled tiers.

Module ID	Module name	Main responsibility	Inputs / outputs	Depends on
M1	Tier 1: Client Presentation	User interaction, biometric capture, QR scanner, and real-time dashboard	In: User inputs & touch events / Out: HTTPS/WSS requests	None
M2	Tier 2: API Gateway & Edge	TLS termination, token-bucket rate limiting, OAuth2/JWT validation, idempotency checks	In: Ingress HTTP/TLS payload / Out: Authenticated & rate-limited	M1

			request	
M3	Tier 3: Client Presentation Layer Proxy	Business rule validation, fee calculation, payment FSM orchestration	In: Validated Gateway request / Out: Transaction state command	M2
M4	Tier 4: Core Double-Entry Ledger	Atomic debit-credit journal postings, serializable ACID transaction execution	In: Transaction state command / Out: Ledger entry status & receipt	M3
M5	Tier 5: Banking & Gateway Adapter	Integration with banking switches, ISO 8583 message formatting, payment webhooks	In: Bank clearing request / Out: Bank authorization / settlement response	M3, M4
M6	Tier 6: Async Event & Notification Hub	Event streaming for audit trails, push notifications, and real-time fraud telemetry	In: System event payload / Out: Async notification & audit streams	M3, M4, M5

Module hierarchy / decomposition diagram:

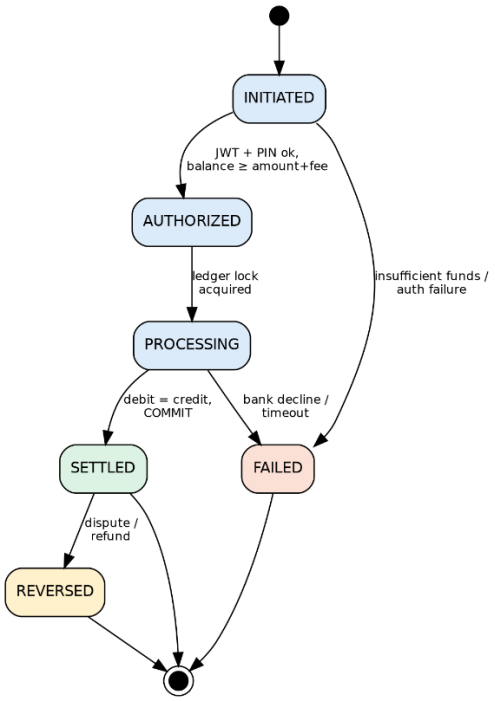


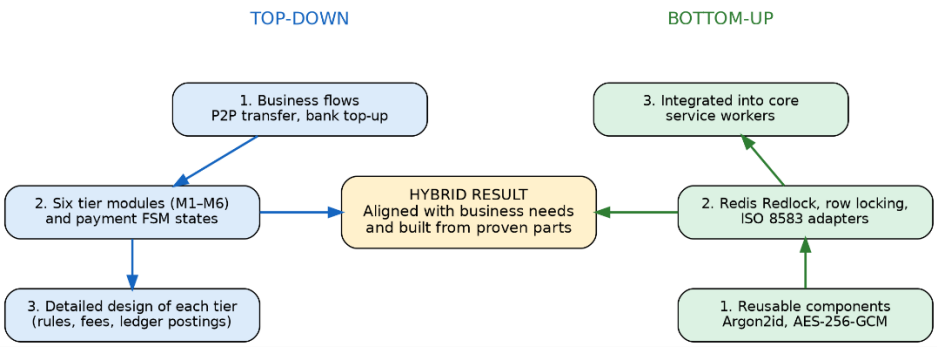
C. Design Strategy

Selected strategy: ☐ Top-Down ☐ Bottom-Up ☑ **Hybrid**

Justification : A hybrid design strategy allows top-down domain modeling of business transaction flows (such as P2P transfers and banking top-ups) while concurrently integrating bottom-up re-usable components (including Argon2id PIN hashing, AES-256-GCM encryption, Redis distributed locks, and PostgreSQL PL/pgSQL double-entry ledger triggers). This dual approach guarantees strict architectural alignment with financial business requirements while leveraging highly resilient, pre-tested security and data infrastructure.

Evidence: Top-down decomposition defines high-level FSM states (INITIATED -> AUTHORIZED -> PROCESSING -> SETTLED). Simultaneously, bottom-up integration incorporates Redis Redlock drivers, PostgreSQL row-level pessimistic locking, and ISO 8583 banking message adapters into core service workers.

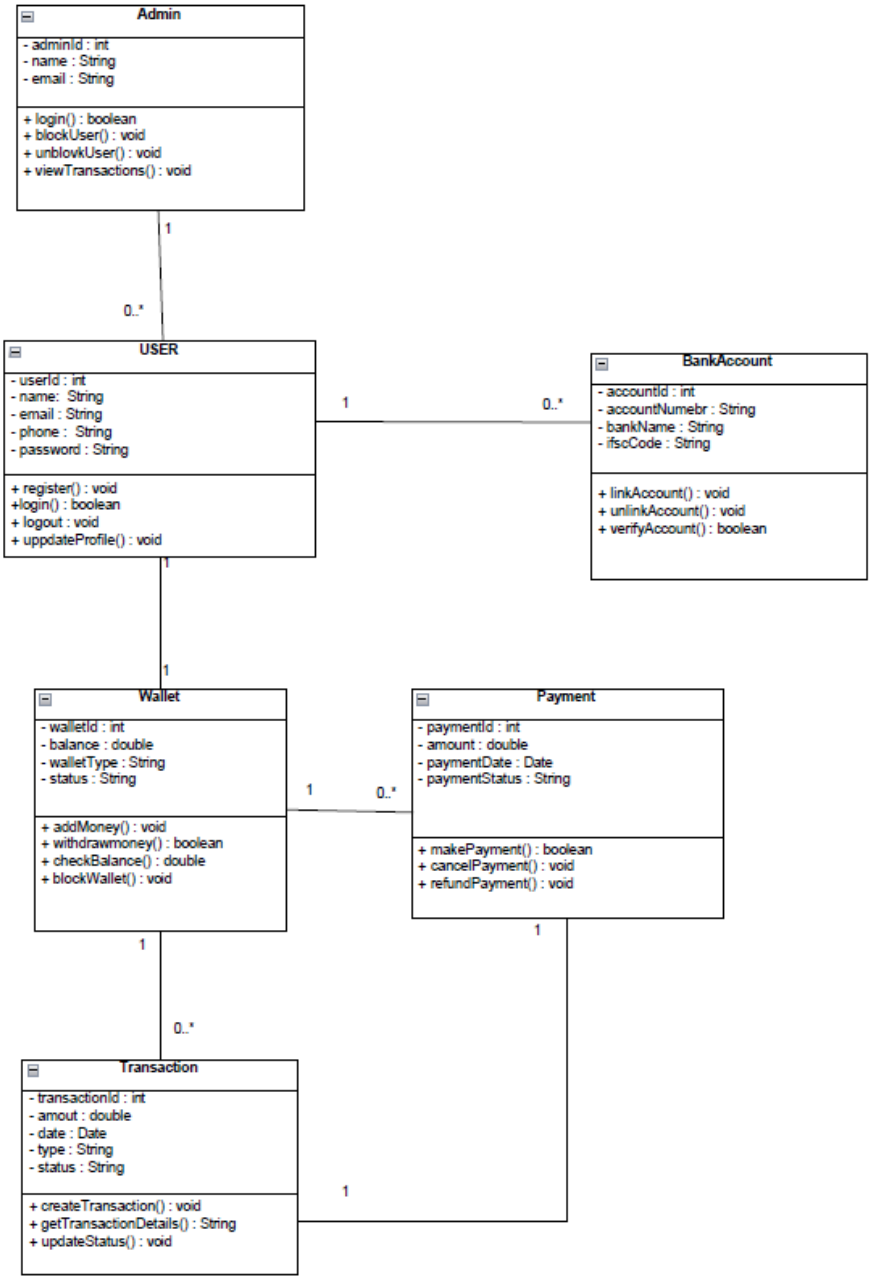




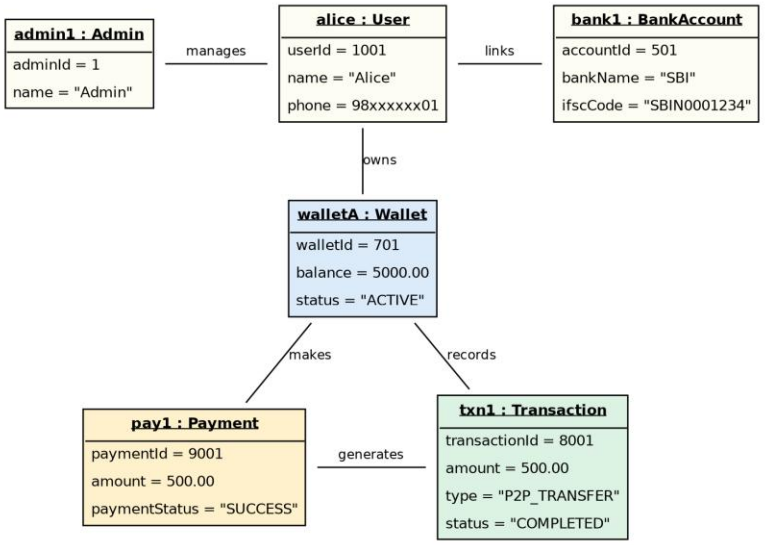
D. Design Approach

Selected approach: ☐ Function-Oriented ☐ Object-Oriented ☑ **Mixed**

Justification : A mixed approach leverages Object-Oriented paradigm concepts to model domain state structures (e.g., Wallet Entities, Journal Entries, and Ledger Postings) while using Function-Oriented streams for stateless processing operations (e.g., Token Bucket rate-limiting functions, HMAC-SHA256 signature verification, and heuristic fraud risk scoring pipelines). Evidence: Object models encapsulate database schemas and business state, whereas stateless functional pipelines execute transaction transforms and asynchronous event broadcasts.



Class diagram



Object Diagram

E. Coupling Analysis

For at least three important module-to-module interactions, identify the information exchanged, dependency and coupling type (where applicable). Aim for low coupling.

Module pair	Interaction / shared data	Coupling type / level	How to reduce dependency
Tier 2 API Gateway <-> Tier 3 Transaction Routing Engine	Passes primitive transaction parameters (Amount, Sender ID, Recipient ID, Idempotency Key)	Data Coupling (Low)	Maintain lightweight DTO schemas over gRPC/REST without exposing internal database structures.
Tier 3 Transaction Engine <-> Tier 4 Core Double-Entry Ledger	Passes composite Journal Entry object containing debit and credit postings data structure	Stamp Coupling (Low-Medium)	Standardize ledger transaction DTOs to avoid passing unneeded user profile data to the database tier.
Tier 3 Transaction Engine <-> Tier 6 Async Event Hub	Passes event execution status flags and transaction control signals for async logging	Control Coupling (Low)	Use publish-subscribe messaging over Apache Kafka to decouple execution from audit analytics.

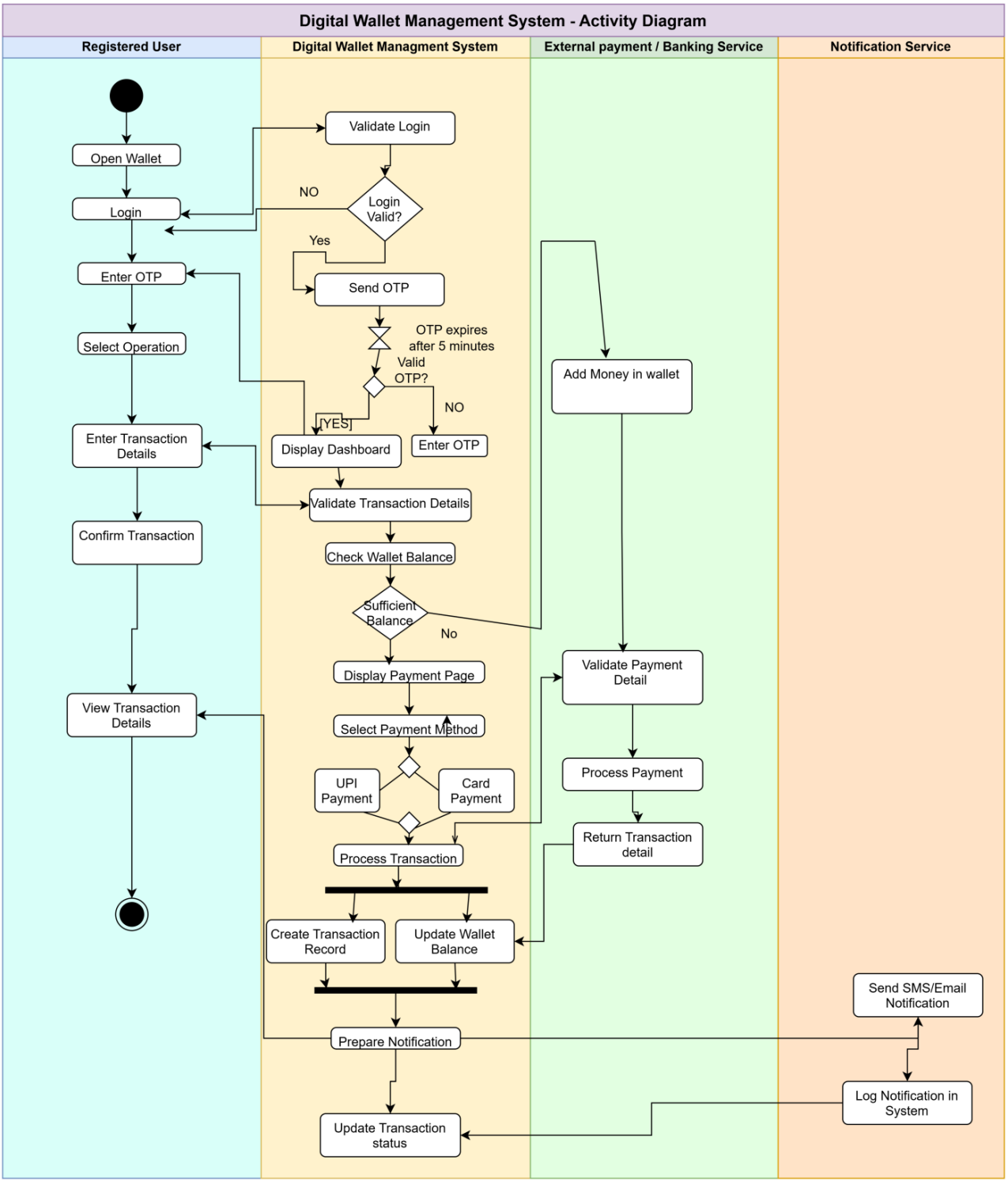
F. Cohesion Analysis

For at least three modules, describe the single main responsibility and evaluate how closely its internal tasks belong together. Aim for high cohesion.

Module	Main responsibility	Cohesion type / level	Improvement, if needed
Tier 4: Core Double-Entry Ledger	Executes balanced debit/credit journal postings ensuring mathematical zero-drift	Functional Cohesion (High)	Isolate ledger posting calculations from account management operations.
Tier 3: Transaction Routing Engine	Evaluates balance, calculates fees, and drives transaction state machine sequentially	Sequential Cohesion (High)	Separate fee calculation rules from core state transition execution.
Tier 5: Banking & Gateway Adapter	Processes clearing requests and converts ISO 8583 banking messages over shared data	Communicational Cohesion (Medium-High)	Refactor gateway webhook handlers into dedicated micro-adapters.

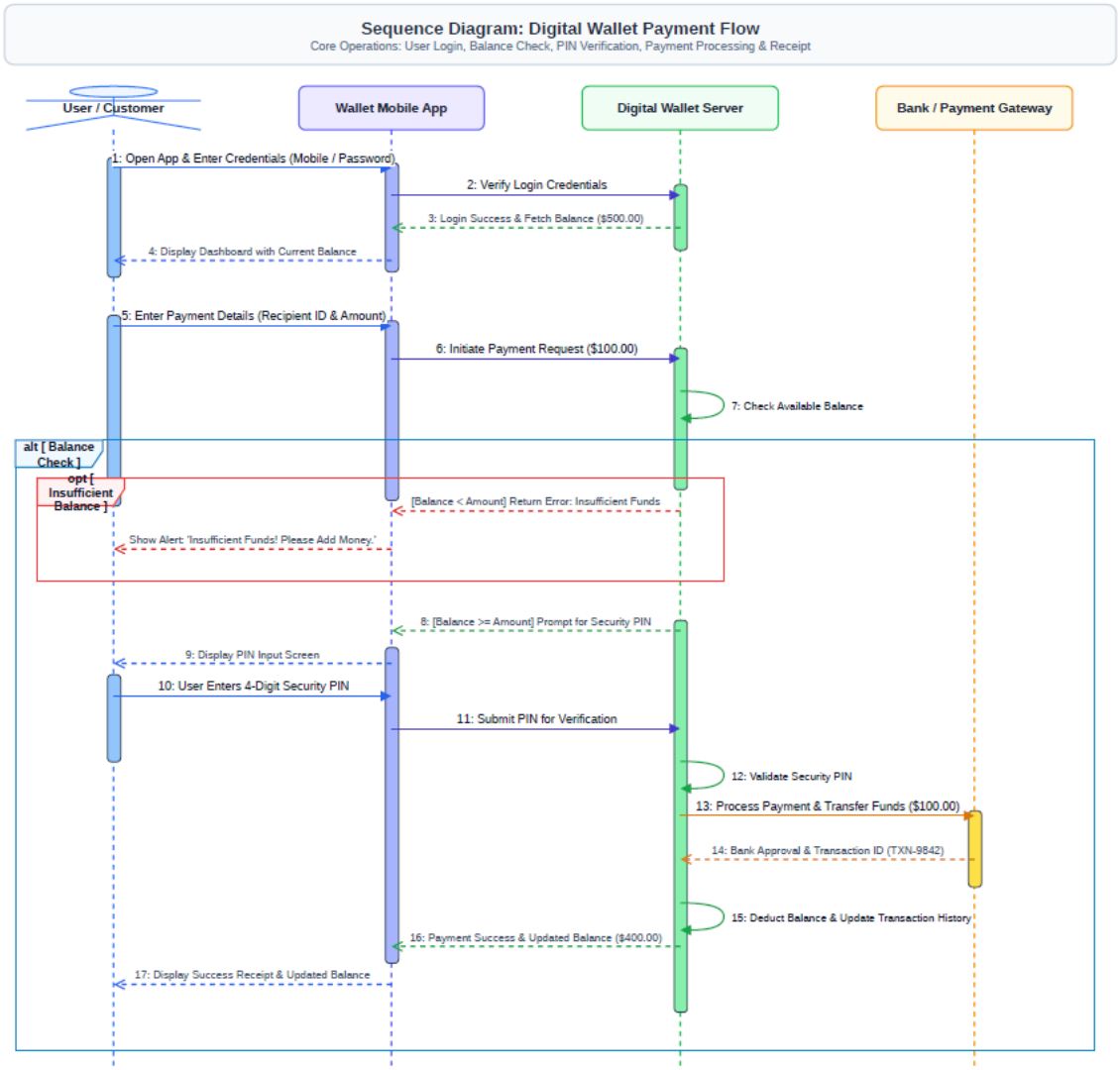
G. Module Interaction and Design Quality

Module Interaction Sequence: [User] -> enters credentials -> [Wallet Mobile App (Tier 1)] -> sends login request -> [Digital Wallet Server (Tiers 2–4)] verifies credentials and returns the balance -> user enters recipient ID and amount -> server checks the balance -> if the balance is too low, an “Insufficient Funds” alert is shown; otherwise the server asks for the security PIN -> app submits the PIN -> server validates it -> server sends the payment request to [Bank / Payment Gateway (Tier 5)] -> bank approves and returns a transaction ID -> server deducts the balance and updates the transaction history -> app shows the success receipt and the updated balance to the user.



Activity Flow: Figure 7 shows the same process from the user's point of view, using four swimlanes. The Registered User lane is the user and the mobile app (Tier 1), the Digital Wallet Management System lane covers the server (Tiers 2–4), the External Payment / Banking Service lane is Tier 5, and the Notification Service lane is Tier 6. After a valid login and OTP check (the OTP expires after 5 minutes), the user enters and confirms the transaction details. The system validates them and checks the wallet balance. If the balance is insufficient, money is added through the banking service; otherwise the payment page is shown and the user pays by UPI or card. The system then processes the transaction, creates the transaction record and updates the wallet balance in parallel, prepares a notification, and updates the final transaction status, which the user can view.

Sequence Diagram:



Sequence Explanation: It shows one complete payment between four participants: the User / Customer, the Wallet Mobile App (Tier 1), the Digital Wallet Server (which covers Tiers 2–4: gateway, transaction engine and ledger) and the Bank / Payment Gateway (Tier 5). Solid arrows are requests, dashed arrows are return messages, and arrows that loop back to the same participant are internal steps. The alt frame marks the balance check, where the flow splits into an insufficient-funds branch and a sufficient-funds branch.

Design Quality:

1. Low coupling. The mobile app, the server and the bank communicate only through simple request and response messages. The app never accesses the database or the bank directly, so any one part can be changed without breaking the others.

2. High cohesion. Each module has one clear job. The mobile app collects input and displays results. The server verifies login and PIN, checks the balance, and records the transaction. The bank only processes the payment.

3. Security. The user is checked twice, first by login (password) and then by a 4-digit PIN before any money moves. Credentials and PIN are sent over HTTPS, and the bank's transaction ID makes every payment traceable.

4. Reliability and correctness. The balance is checked before payment, so the user cannot overspend. The wallet balance is updated only after the bank approves, so the wallet and bank

records stay consistent.

5. Maintainability and scalability. The bank is reached through one request and response pair, so a new bank or payment provider can be added without changing the app. The server can be scaled by running several copies.

6. Risks and improvements. If the server fails after the bank approves but before the wallet is updated, the records may differ, so each payment should carry an idempotency key and be recovered or reversed automatically. The bank call needs timeouts and retries, and repeated wrong PINs should lock the wallet after a few attempts.

Design Quality Assessment: The two diagrams show a design that is easy to follow, secure and reliable.